

Summary of Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates

Vishnu Kannan (vishnuka), John Xie (johnxie), Albert Cao (caoalb), Eric Zhao (zhaoeric)

Problem and Motivation

Training deep neural networks at scale requires hybrid parallelism, which uses data, pipeline, and tensor parallelism to organize models with hundreds of billions of parameters across clusters of GPUs. These jobs run on hundreds of machines for long periods of time, making hardware failures inevitable. Because of the synchronous nature of distributed training, a single node failure can force all GPUs to idle until it recovers, which can lead to underutilization.

Existing solutions to this problem fall at one of two extremes. On one side, checkpoint-based systems (which periodically save the model state) reduce compute overhead, but result in significant pauses in training as large amounts of data need to be loaded upon failure. On the other hand, redundant computation techniques (which duplicate state/computation in multiple nodes) allow for faster recovery, but result in significant compute and memory overhead.

Ultimately, as model size continues to increase, there is a pressing need for a fault-tolerant training framework that maintains high throughput and scalability, without the overhead and recovery times tradeoffs that come with the existing frameworks.

Related Works

Varuna

- Checkpoints model state periodically. When a node fails, it reconfigures the training job to optimally use the remaining GPUs and restarts from the most recent checkpoint.
- **Limitations:** Varuna results in low throughput when failures occur frequently due to the time intensive checkpoint loading process.

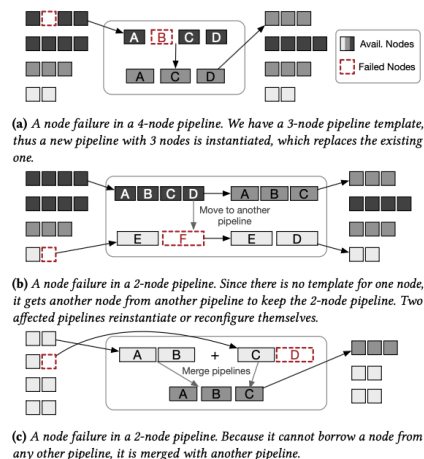
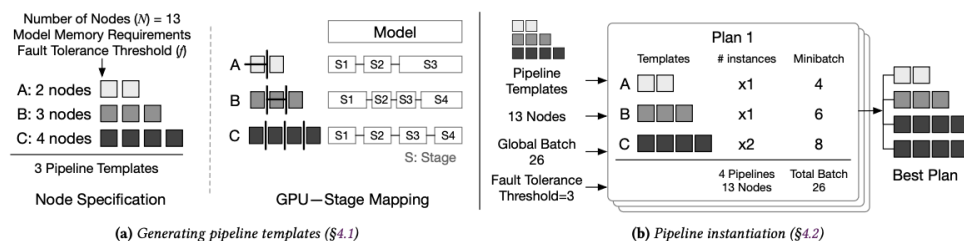
Bamboo

- Duplicates each pipeline stage on adjacent nodes. When a node fails, its backup can take over immediately without pause.
- **Limitations:** Faces out of memory issues with larger models, as each node is required to hold roughly twice as much state. Also, if adjacent nodes fail, model state will be lost, and a reconfiguration and restart from a checkpoint is needed.

Solution Overview

Oobleck is a fault-tolerant distributed training framework that allows for elastic, high-throughput training of large DNNs in the presence of node failures. Before training, Oobleck runs a planning phase that generates heterogeneous templates, each of which is a valid pipeline partitioning using a different number of nodes. In the presence of up to f simultaneous failures, Oobleck

generates templates in a way such that it guarantees a linear combination of these templates can fully utilize the remaining nodes. The combination of templates to instantiate are chosen using a dynamic programming algorithm to maximize throughput. Within each template, a divide and conquer algorithm is used to determine the optimal partitioning of the model across the available nodes. Since the templates are heterogeneous, Oobleck’s batch distributor assigns microbatches to templates in a way that balances latency across all instantiations.



When a node fails, Oobleck attempts three steps in order. First, it attempts to find a suitable template with the nodes remaining from the original template. Second, if no such template exists, the current pipeline will borrow nodes from another pipeline and re-instantiate both pipelines with the newly selected templates. Finally, if borrowing is not an option, Oobleck merges existing pipelines into larger ones. In general, when re-instantiating pipelines, Oobleck copies missing model layers from non-failed replicas.

Oobleck was evaluated against Varuna and Bamboo in two settings (one with controlled node failures and one with spot nodes). In general, Oobleck achieved much higher throughput than both alternatives, especially for large models. Bamboo is unable to support models above a certain size, as it requires nodes to hold twice as much state as normal. Varuna also struggles with large models, as checkpoint loading becomes extremely slow.

Limitations

Oobleck’s design makes several assumptions that limit its generality. It assumes homogeneous GPU hardware and thus can produce an unoptimal partitioning in scenarios when hardware is heterogeneous. The latency of the planning phase also grows quickly with model size (see Table 3), making template generation potentially impractical for very large models. Finally, in order to handle heterogeneous templates, Oobleck performs layer-wise synchronization for gradients, as opposed to stage-wise synchronization. This adds additional communication overhead, which could scale poorly with large models and cause additional issues.

Future Research Directions

The paper uses throughput as the primary metric when evaluating Oobleck against Varuna and Bamboo. Future research could explore Oobleck's impact on net training time (including the template generation phase) and model convergence. It would also be interesting to explore how the latency of the template generation phase changes based on factors such as the model size, model architecture, and the number of available GPUs.

Summary of Class Discussion

Q: How does Oobleck work if the GPUs are not homogenous?

A: Oobleck does not handle this scenario, but it should (the authors said they want to expand on this). This big companies they talked to said their GPU fleets are usually homogenous, meaning this is not a problem. However, smaller companies with smaller, heterogeneous fleets are more interested in such a solution. A group actually explored this idea (extending Oobleck to work for heterogeneous fleets) for their course project a couple semesters ago, but stopped working on it when they graduated.

Q: Does Oobleck use profiling, and if so, how?

A: Yes. For example, Oobleck profiles the different templates to estimate their throughputs. These results are used during recovery, as Oobleck seeks to find the linear combination of templates that maximizes throughput.

Q: What if the system has the resources to run one replica (as opposed to $f + 1$)?

A: If a system does not have enough resources to run multiple pipelines, its not a viable use case for Oobleck. In this case, you can deploy one replica of the model, partitioned amongst the available GPUs, and use traditional checkpointing to recover after failures.

Q: How are states recovered when there is a failure?

A: For the dataset, Oobleck stores metadata to indicate which examples have been processed. This metadata ensures that the same example is never trained on twice. For model states, the fact that we have $f + 1$ replicas means the complete model state will always be reconstructable for the remaining replicas, as long as we don't have more than f simultaneous faults.

Q: Can you explain how the following transition works? Assume we have templates for $n = 3$ nodes and $n = 4$ nodes? We have 1 instantiation of the 3 node template and 1 instantiation of the 4 node template. One of the nodes in the 3 node instantiation fails. What happens next?

A: When one node fails in the 3 node instantiation, there are only 2 nodes remaining. Since no template exists for $n = 2$, Oobleck borrows a node from the 4 node instantiation. This creates two **new** 3 node instantiations. The original 4 node instantiation will be transitioned to a 3 node pipeline, and the original 3 node instantiation will incorporate the newly added 3rd node.

Q: Does Oobleck discuss what happens if another failure happens during recovery?

A: This is an interesting question. If the second failure happens before the **implementation** of the reconfiguration, you can abort and reconfigure. If it occurs during the implementation of the reconfiguration, that is a trickier problem, as there can be some interesting consistency issues.

Q: What counts as a node failure? Is it only complete failure, or do performance degradations count too? What mechanism does Oobleck use to detect these failures?

A: Oobleck launches a process on each node, which establishes a TCP connection with some centralized node. When the node fails, the socket is broken, making the centralized node aware of the failure. It can then trigger a reconfiguration. However, this isn't the best solution, as the centralized node becomes a single point of failure for the system.

Summary of Tenplex: Dynamic Parallelism for Deep Learning using Parallelizable Tensor Collections

Vishnu Kannan (vishnuka), John Xie (johnxie), Albert Cao (caoalb), Eric Zhao (zhaoeric)

Problem and Motivation

Today's deep learning jobs are deployed with a fixed set of GPUs and a static multi-dimensional parallelization strategy decided before training time. This becomes a problem as models grow in size, and training takes many days or even weeks. During these periods of time, schedulers can add/remove GPUs to balance utilization or transfer them between jobs to reduce fragmentation. In addition, GPU failures can force training jobs to continue with less resources. Unfortunately, current deep learning systems only adapt to these sorts of changes at the data parallelism level, but are not capable of making adjustments at the pipeline or tensor parallelism levels.

This is problematic as a parallelization configuration that worked well for one allocation may not be optimal for another. The task of finding a new optimal configuration can be outsourced to a tool like Alpa, but the process of implementing this configuration is tricky. If done incorrectly, reconfiguration can affect convergence by creating inconsistent datasets or hyperparameters across workers. Additionally, naive reconfiguration can be expensive, since large volumes of data must be transferred between GPUs or to and from centralized storage.

Tenplex is a solution to this problem. It introduces Parallelizable Tensor Collection (PTC), which is an abstraction that externalizes job state and allows it to be transformed in an efficient and non-decentralized manner whenever GPU allocations change. Evaluations show that Tenplex leads to faster recovery times after failures, without affecting model convergence or throughput.

Related Works

Model Libraries (Megatron-LM, Deep Speed)

- Distributed deep learning frameworks for training models, designed to scale to large models by combining data, pipeline, and tensor parallelism.
- **Limitations:**
 - Only supports runtime scaling via the data parallelism dimension (batch size for each data parallel worker is adjusted).
 - Limited to specific architectures like Transformer models.

Elastic ML Systems (Elastic Horovod, Torch Distributed Elastic, Varuna, KungFu)

- Provide APIs/interfaces to save/access model checkpoints and broadcast model state when GPU resources change.
- **Limitations:**
 - These frameworks provide tools to redistribute state among workers, but they do not actually implement it. Implementing state redistribution is tricky and can lead to inconsistencies that affect convergence, as discussed above.

- Does not support multi-dimensional data parallelism.

Virtual Device Layers (Singularity, VirtualFlow, EasyScale)

- Makes deep learning jobs device independent by virtualizing GPUs and dynamically updating the virtual to physical resource mapping at runtime.
- **Limitations:** Similar to the previous related works, these approaches do not support runtime changes at the pipeline or tensor parallelism levels.

Solution Overview

Tenplex introduces a new abstraction called PTC to support the dynamic reconfiguration of deep learning jobs when GPU resources change at runtime. PTC is a hierarchical, file-system-like representation of the job's state. This includes the partitioned dataset (with metadata to track which examples have been trained on so far), as well as the current model parameters.

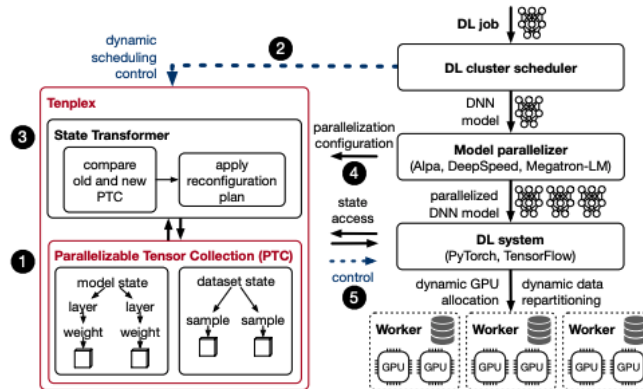


Fig. 5. TENPLEX design

When the cluster scheduler adds or redeployes GPUs, it notifies Tenplex, which requests a new optimal parallelization configuration from a model parallelizer like Alpa or Megatron-LM. The **State Transformer** on each worker then computes the minimal sequence of operations (split, re-partition, merge) to transform from the existing PTC to the new PTC. This decentralized approach to reconfiguration is one of Tenplex's key contributions, as existing frameworks typically rely on centralized approaches that create bottlenecks/single points of failure. Each worker uses an in-memory **Tensor Store** that holds its local dataset and model partitions. Based on the reconfiguration plan, workers fetch the required sub-tensors from either their own Tensor Store or from remote workers and continue training under the new configuration.

Tenplex has been shown to greatly reduce reconfiguration time. When scaling up from 8 to 16 GPUs, Tenplex is 24% faster than DeepSpeed and 10% faster than Singularity. When scaling down from 16 to 8 GPUs, Tenplex is 64% and 43% faster than these alternatives, respectively.

It also achieves roughly similar throughput when compared to Horovod (a distributed training library **without** support for dynamic reconfiguration) and Horvod Elastic (which only supports reconfiguration at the data parallelism level). Tenplex also preserves model convergence, by ensuring dataset and hyperparameter stability across reconfigurations.

Limitations

1. Tenplex uses parallelizers like Alpa and DeepSpeed to generate the new parallelization plans. These parallelizers are time-intensive and designed to run offline, prior to training. Using them while training can cause device idling/severe underutilization.
2. It is unclear how Tenplex handles subtle node failures (i.e. not fail stop). In such cases, is it the responsibility of the DL scheduler to detect the failure, or does Tenplex have a mechanism to handle this?
3. Tenplex was evaluated on clusters with at most 32 GPUs, but state-of-the-art models are typically trained on clusters that are several orders of magnitude larger. It is possible that the latency and cost of reconfiguration increase disproportionately at this scale.

Future Research Directions

Future research could try to find an alternative to using time-intensive parallelizers like Alpa and DeepSpeed during training. It could also look into supporting additional forms of parallelism, like context or sequence parallelism, and explore whether the redistribution of state at these levels could be implemented using the existing split, partition, and merge operations.

Summary of Class Discussion

Q: When Tenplex produces a reconfiguration plan, does it find the optimal partition given the available nodes?

A: Yes, Tenplex uses parallelizers like Alpa or DeepSpeed to produce the new optimal partition. It also generates the cheapest/minimal sequence of operations (split, partition, and merge) to transition from the old PTC to the new PTC. “Cheapest” here refers not just to the number of operations, but also the total amount of data transferred and communication costs.

Q: When there is a GPU failure, how does Tenplex find the next best configuration?

A: See above. Tenplex uses model parallelizers like Alpa and DeepSpeed to generate the best configuration. This configuration is implemented via state transformations.

Q: Can we reallocate the GPUs without pauses in training? For example, if one data parallel worker fails, can the other data parallel workers continue their work?

A: Technically yes, but it wouldn’t provide much real benefit. The data parallel workers can only proceed to the synchronization step once all workers finish their batch. If one worker fails, the others may complete their batches, but they will still have to wait for the failed worker to recover and synchronize gradients before moving on to the next batch.